

Doubly Linked List – Delete From End And Time Complexity.

Program :

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

```

temp->prev->next = temp->next;
if (temp->next != nullptr)
{
    temp->next->prev = temp->prev;
    temp->next = nullptr;
}
temp->prev = nullptr;

cout << "Deleted " << temp->data << " from position " <<
pos << "." << endl;
free(temp);
temp = nullptr;
}

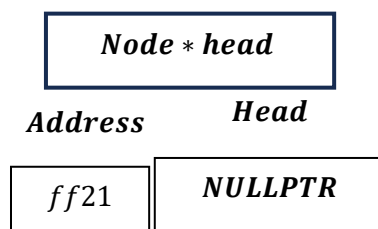
...

case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

Program Analysis:

A. When LIST is Empty



Then, it will print statement : `` List is empty. Nothing to delete.``

And, return statement will exit from the function.

B. When $POS < 1$, As we consider POS starts from `1` – indexing

```
if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}
```

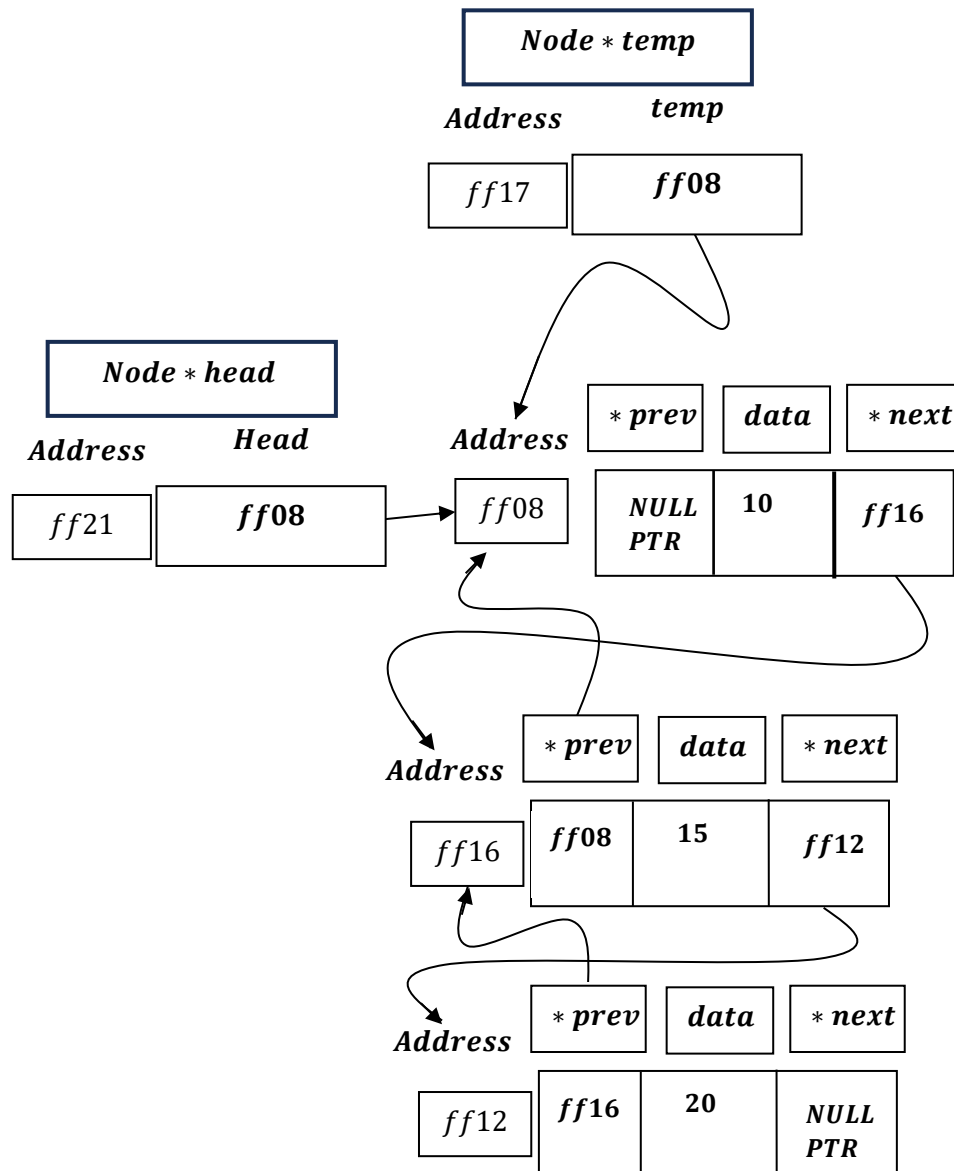
If $pos < 1$, then it prints Invalid Position and return statement, exits from function.

C. When $POS = 1$, then run delete from beginning function.

```
if (pos == 1)
{
    deleteFromBeginning();
    return;
}
```

As, $pos = 1$, deleteFromBeginning() function will run, the working of the function already discussed earlier, the return statement will exit from the function.

```
Node *temp = head;
```



```

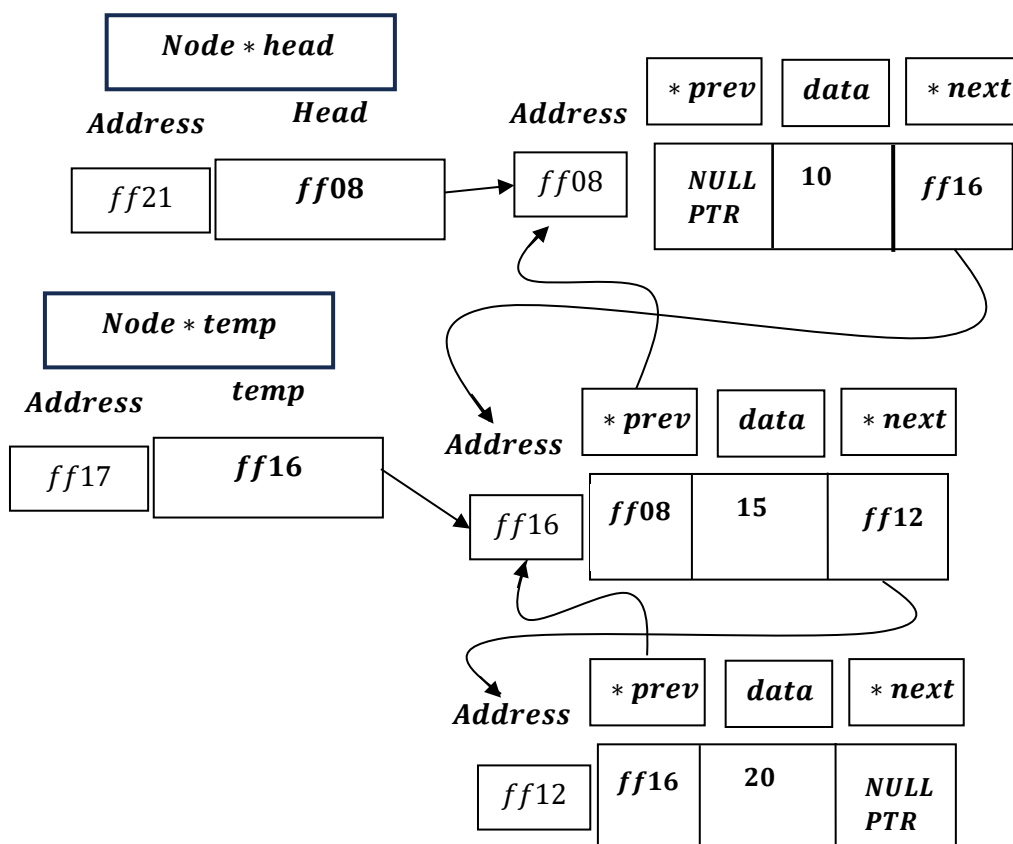
for (int i = 1; i < pos && temp != nullptr; i++)
{
    temp = temp->next;
}

```

Let pos = 2,

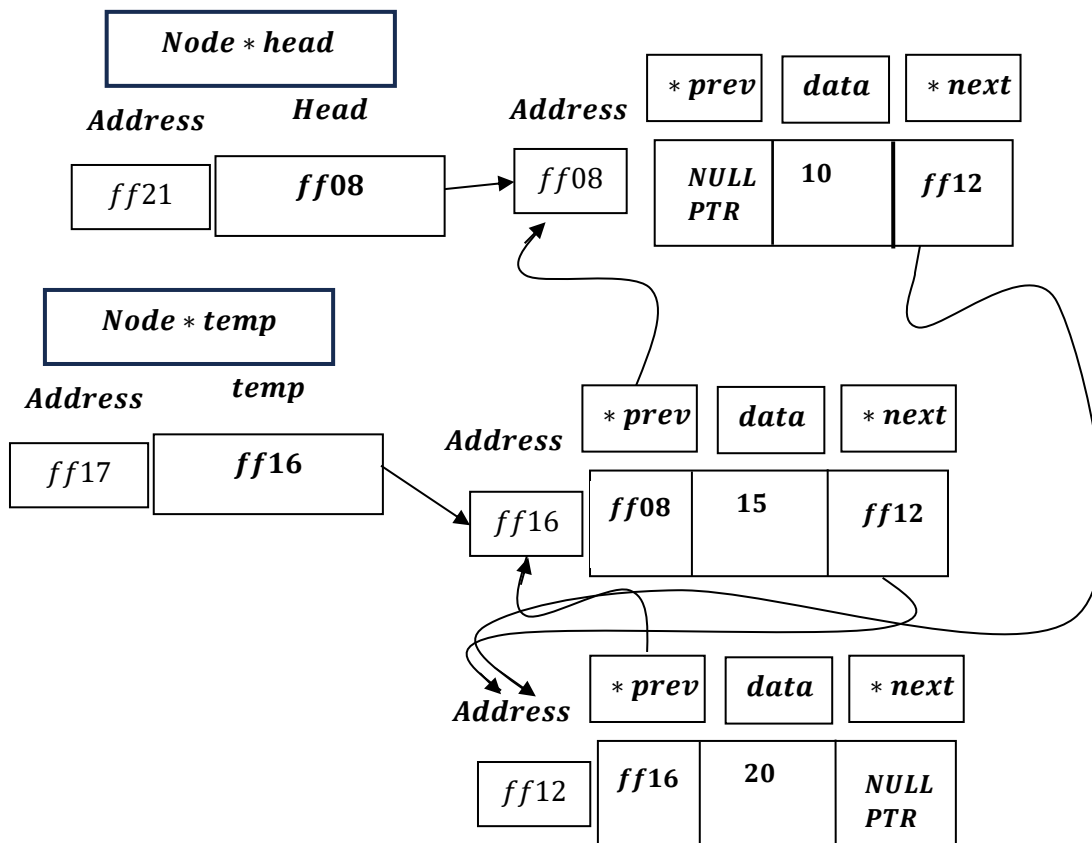
i = 1; i = 1 < pos = 2 is true and temp ≠ nullptr is true ,then :

temp = temp → next = ff16 ;



i = 2; i = 2 < pos = 2 is False and temp ≠ nullptr is true , ⇒ False Condition.

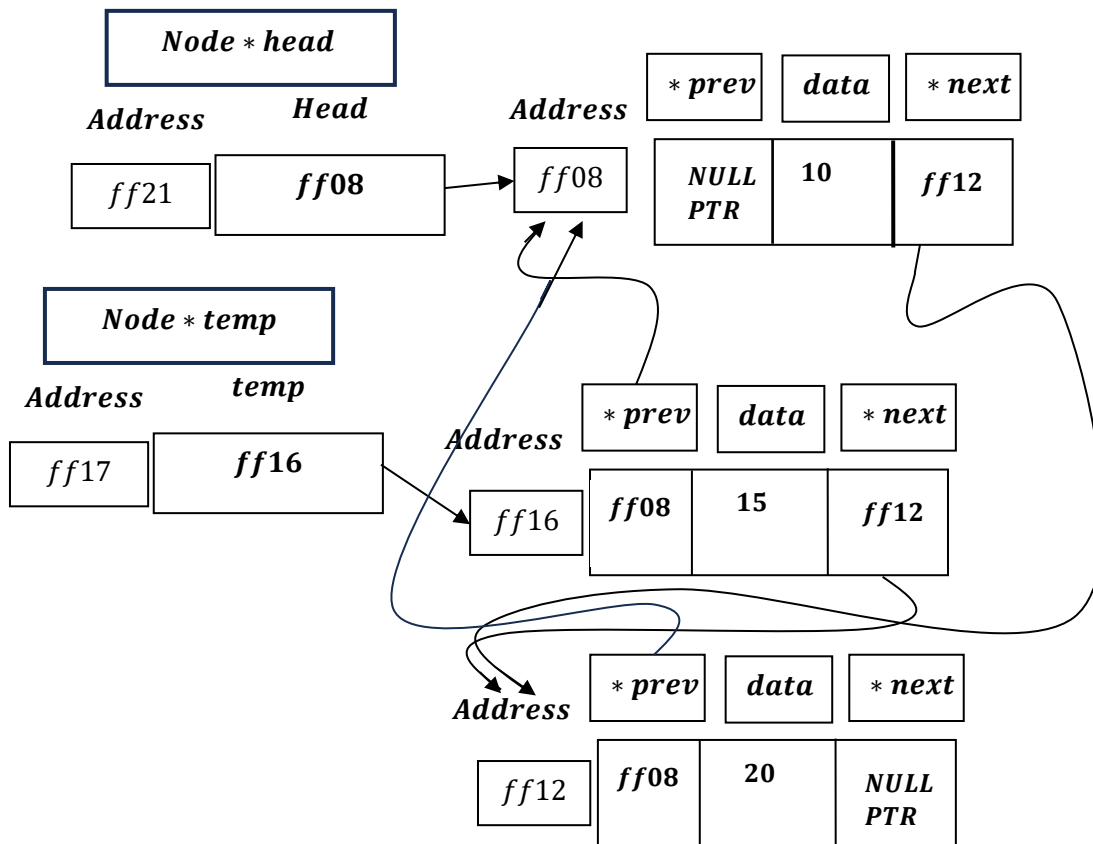
```
temp->prev->next = temp->next;
```



Next,

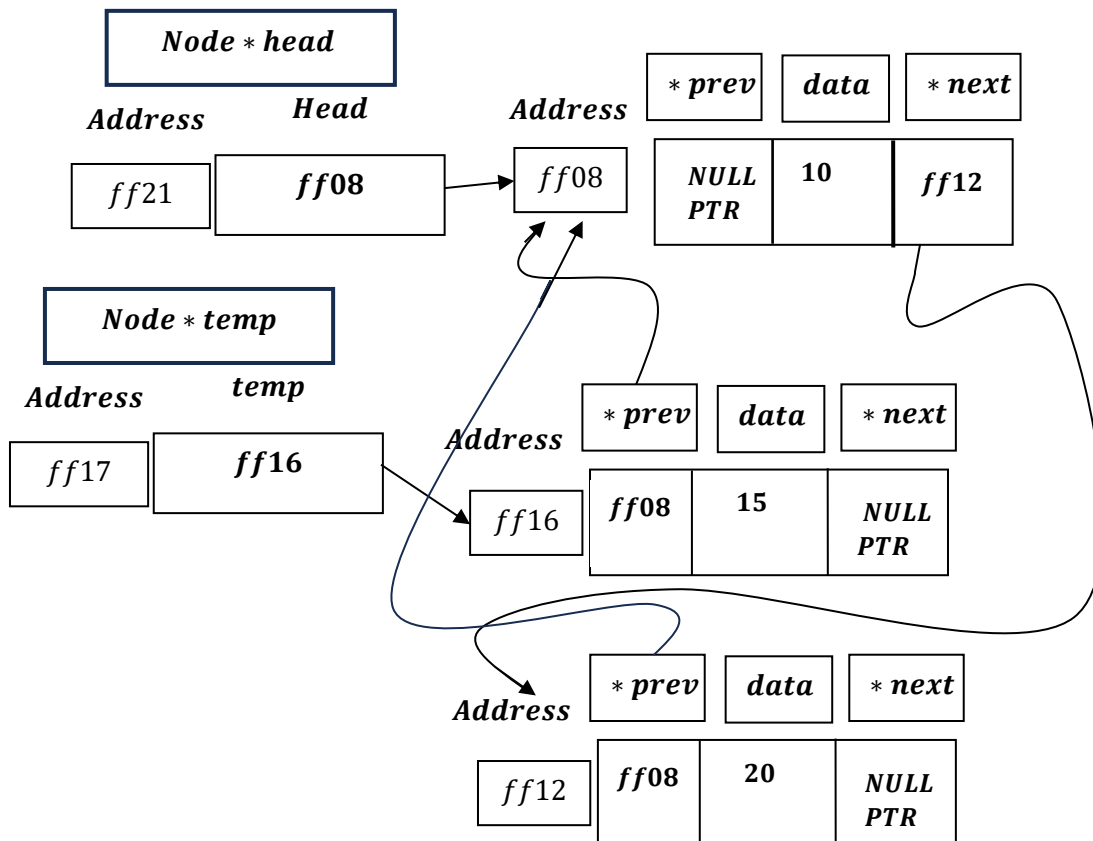
```
if (temp->next != nullptr)
{
    temp->next->prev = temp->prev;
    temp->next = nullptr;
}
```

i. e., $temp \rightarrow next \rightarrow prev = temp \rightarrow prev = ff08$;



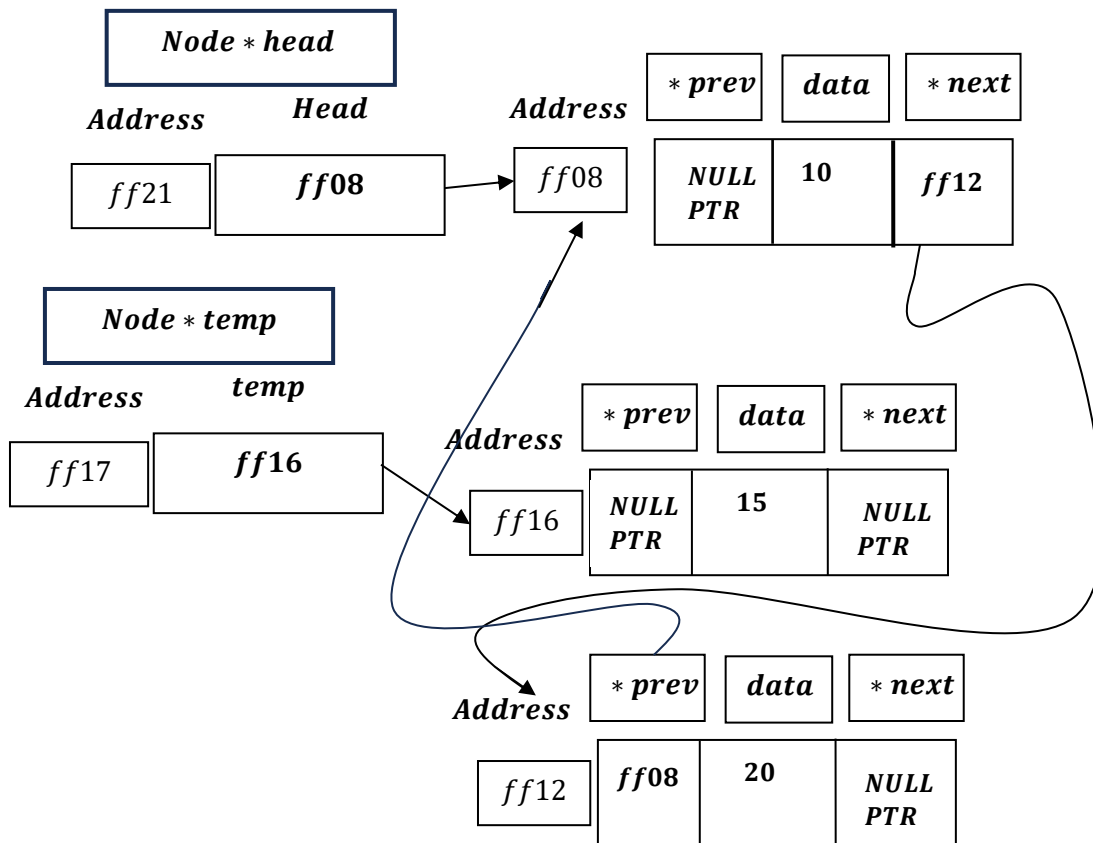
Next,

$temp \rightarrow next = nullptr;$



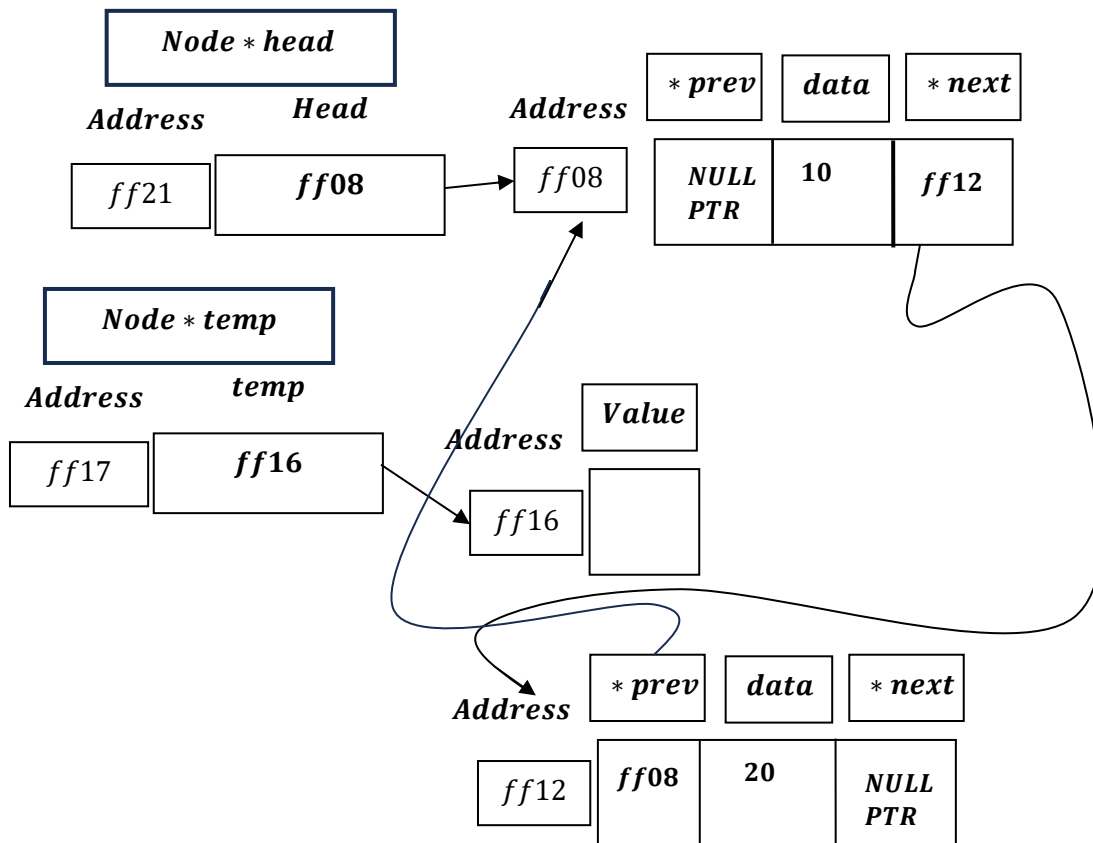
Next,

```
temp->prev = nullptr;
```

Next,

```
free(temp);
```

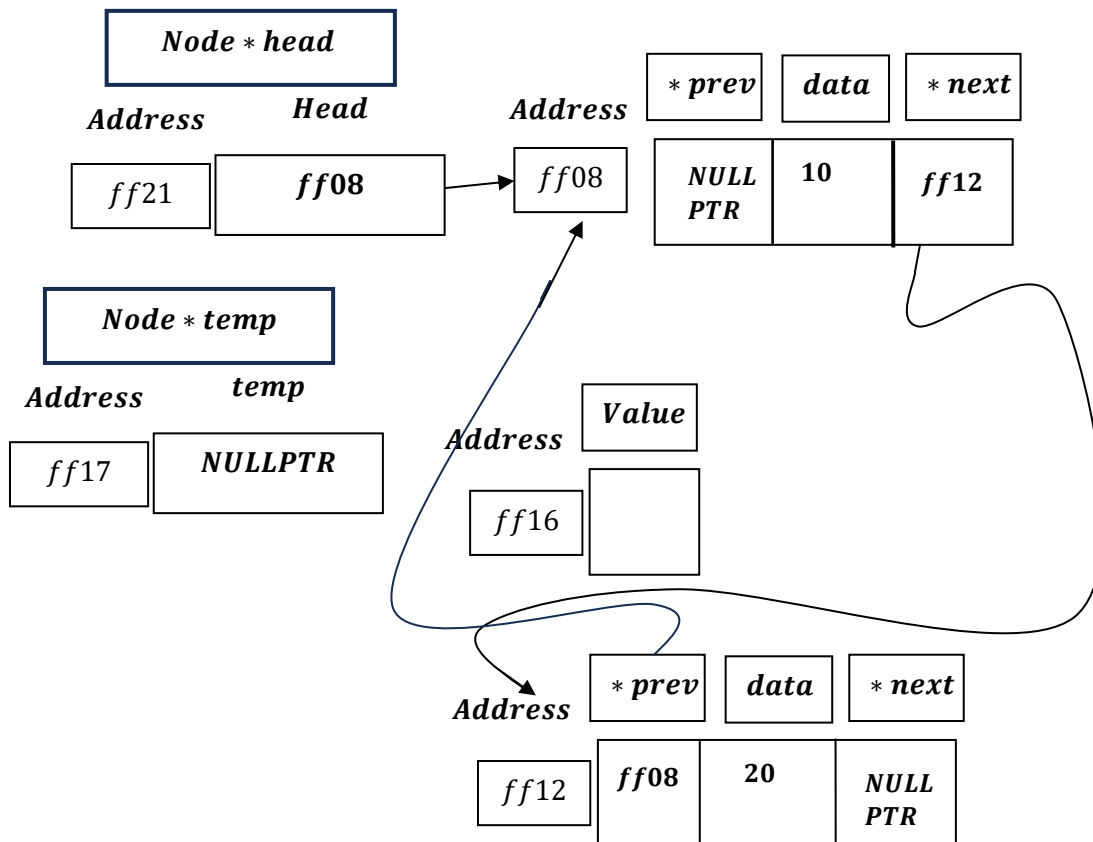


Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

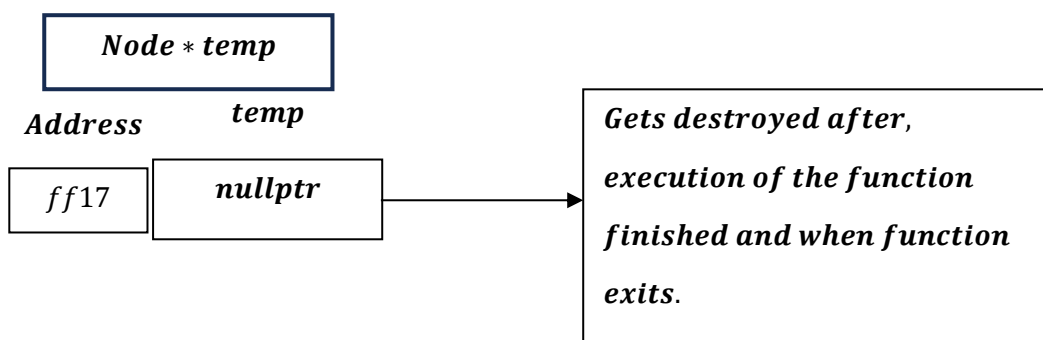
Next,

```
temp = nullptr;
```

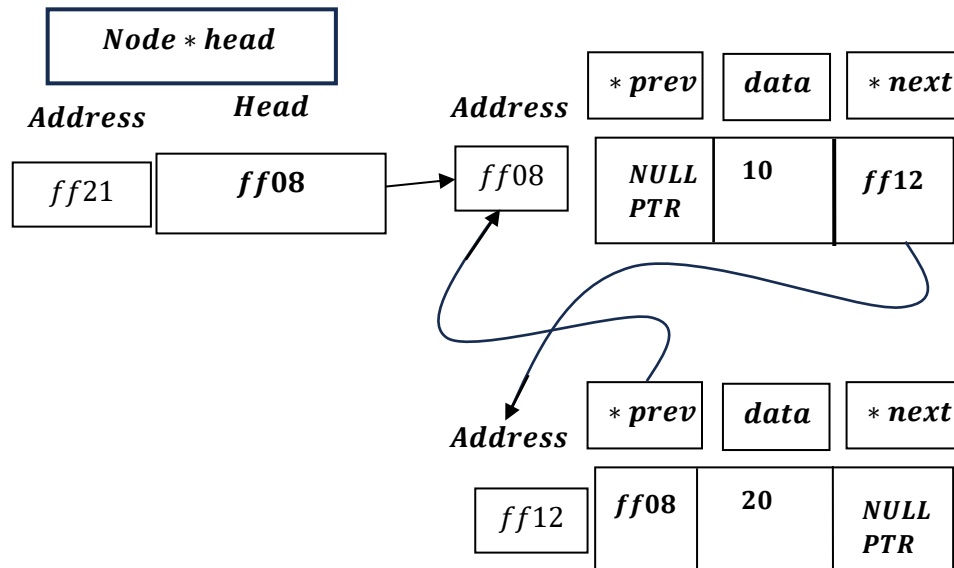


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*

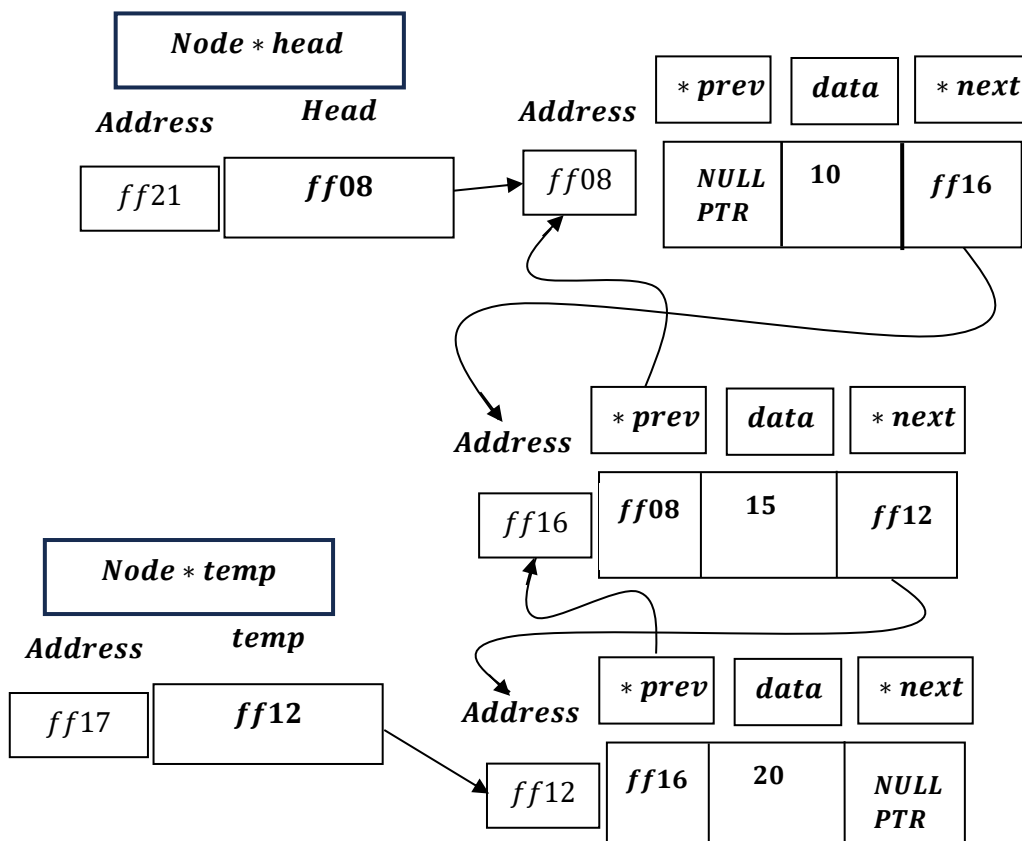


Hence , we are left with:



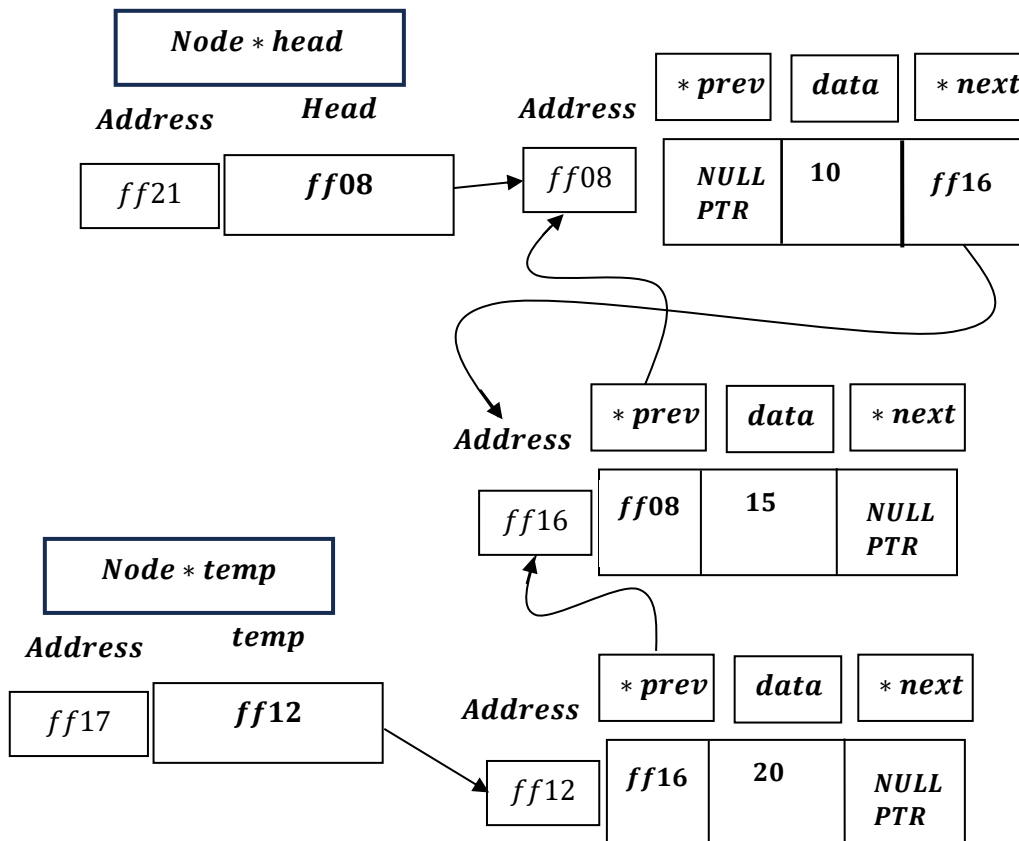
If we want to delete last node:

Let $pos = 3, i = 2; i < pos = 3$ and $temp \neq nullptr$ is true , then :



$i = 3 < pos = 3$ is false and $temp \neq nullptr$ is false, hence exit from function.

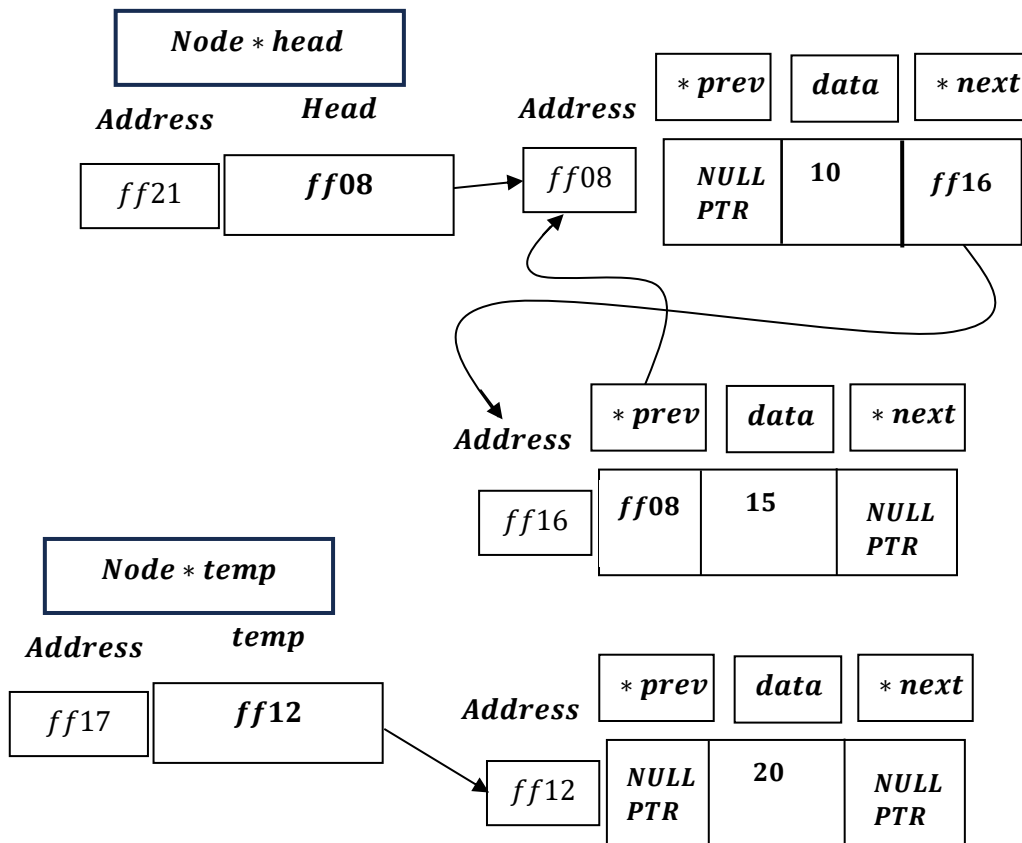
```
temp->prev->next = temp->next;
```



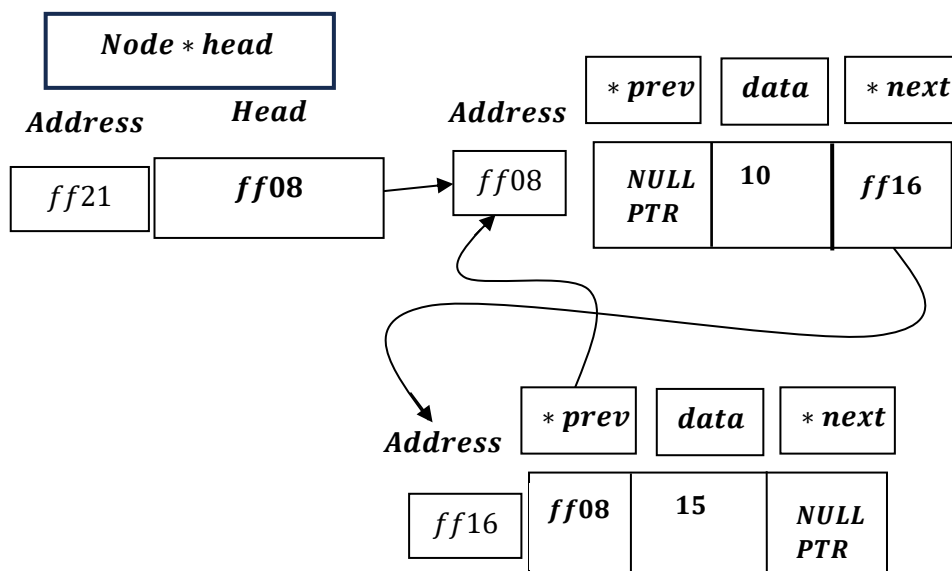
```
if (temp->next != nullptr){.....}
```

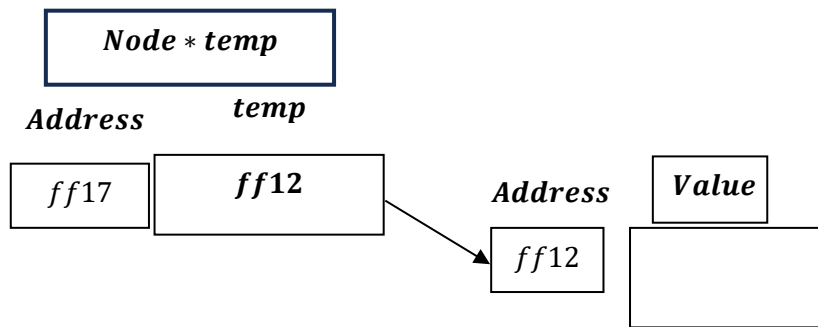
This section of if section will not run as $temp \rightarrow next \neq nullptr$; condition fails.

```
temp->prev = nullptr;
```

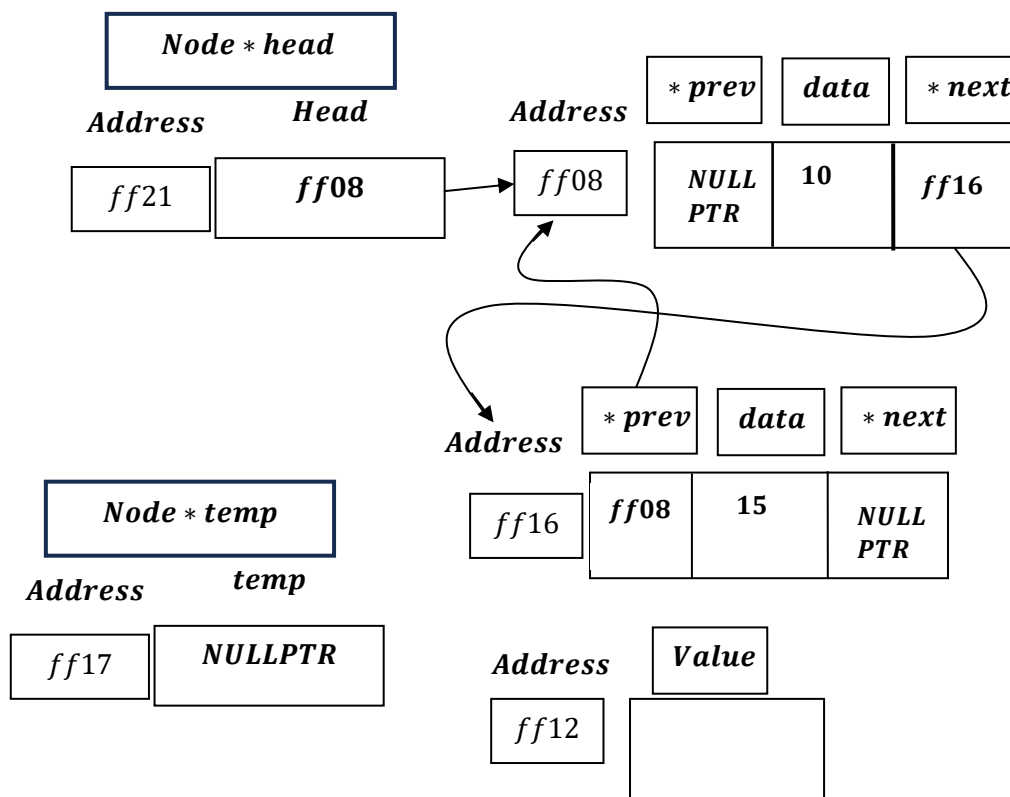


```
free(temp);
```

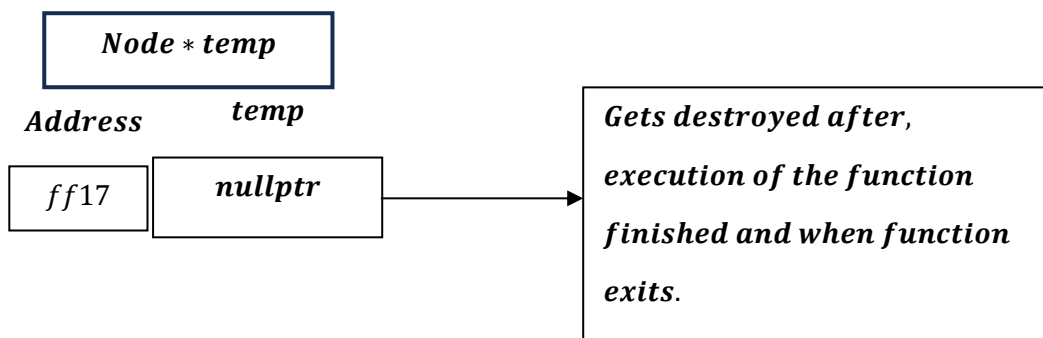




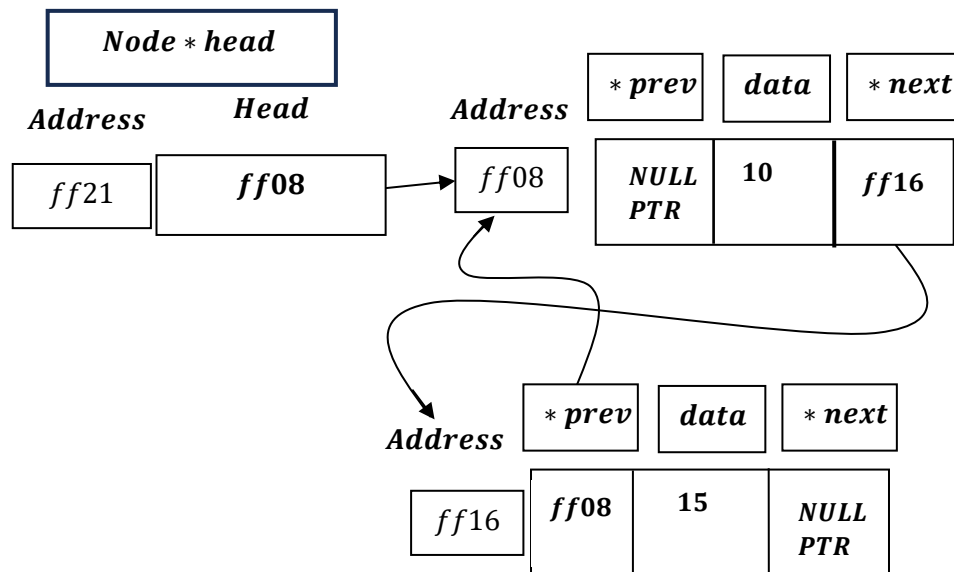
```
temp = nullptr;
```



*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*



Hence ,we are left with:

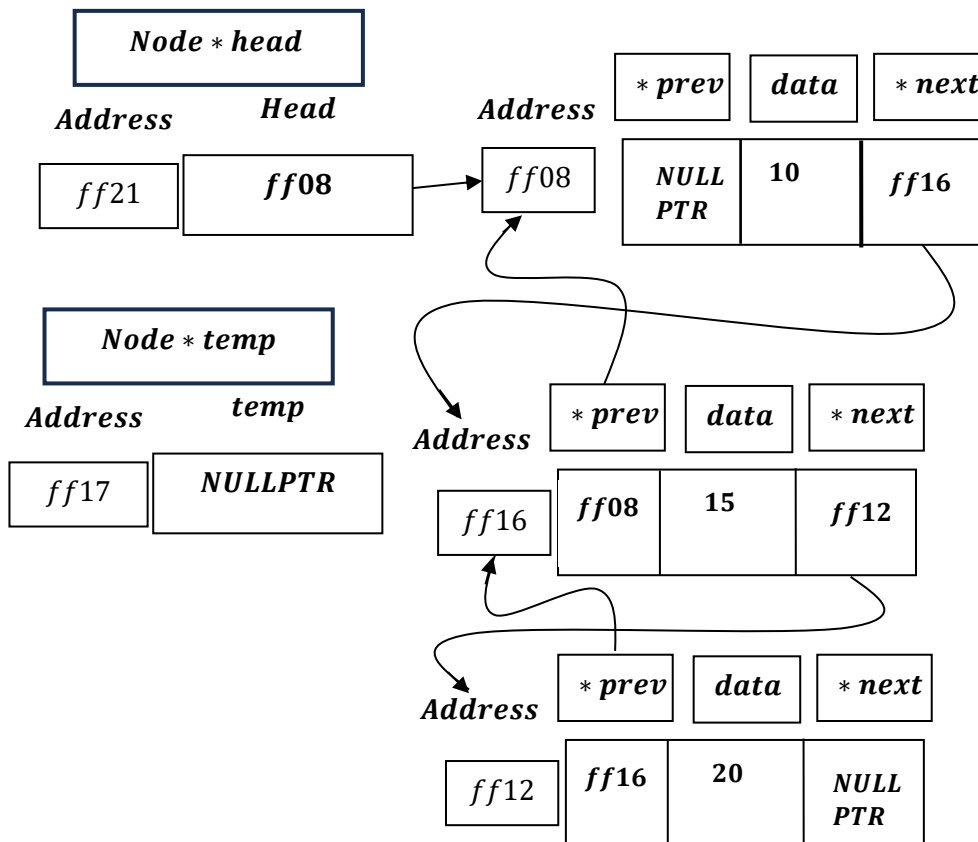


```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

When this runs ?

```
for (int i = 1; i < pos && temp != nullptr; i++)
{
    temp = temp->next;
}
```


Lets say , $pos = 4$, $i = 3$; $i < pos = 4$ i.e. $i = 3$ and $temp \neq nullptr$ is true , then :



$i = 4 < pos = 4$ i.e. $i = 3$ is false and $temp \neq nullptr$ is false, hence exit from function.

And as position is given by user : ``Out of Bounds`` i.e.4.

```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

The if block runs successfully and return statement exits from the function.

Doubly Linked List – Delete From Position [Working Summary]:

This function deletes a node from a specific position in a doubly linked list.

How it Works

1. Handles Edge Cases:

- It first checks if the **list is empty** (`head == nullptr`) or if the **position is invalid** (`pos < 1`).
- If the position is 1, it simply calls the `deleteFromBeginning()` function to handle the deletion and then stops.

2. Finds the Node:

- It starts a temporary pointer `temp` at the `head`.
- A `for` loop traverses the list, moving `temp` one step at a time until it reaches the desired `pos` or the end of the list.

3. Validates Position:

- After the loop, it checks if `temp` has become `nullptr`. If it has, it means the given `pos` was greater than the number of nodes in the list (out of bounds).

4. Detaches the Node:

- `temp->prev->next = temp->next`; This is the crucial first step. It tells the node *before* `temp` to skip over `temp` and point directly to the node *after* `temp`.
- `if (temp->next != nullptr)`: It checks if `temp` is not the last node.
- `temp->next->prev = temp->prev`; If it's not the last node, it tells the node *after* `temp` to point its `prev` link back to the node *before* `temp`, completing the "bypass."

5. Frees Memory:

- The links of the `temp` node (`prev` and `next`) are set to `nullptr` to fully isolate it.
 - `free(temp)` deallocates the memory for the detached node.
 - `temp = nullptr` sets the pointer to null to prevent it from becoming a dangling pointer.
-

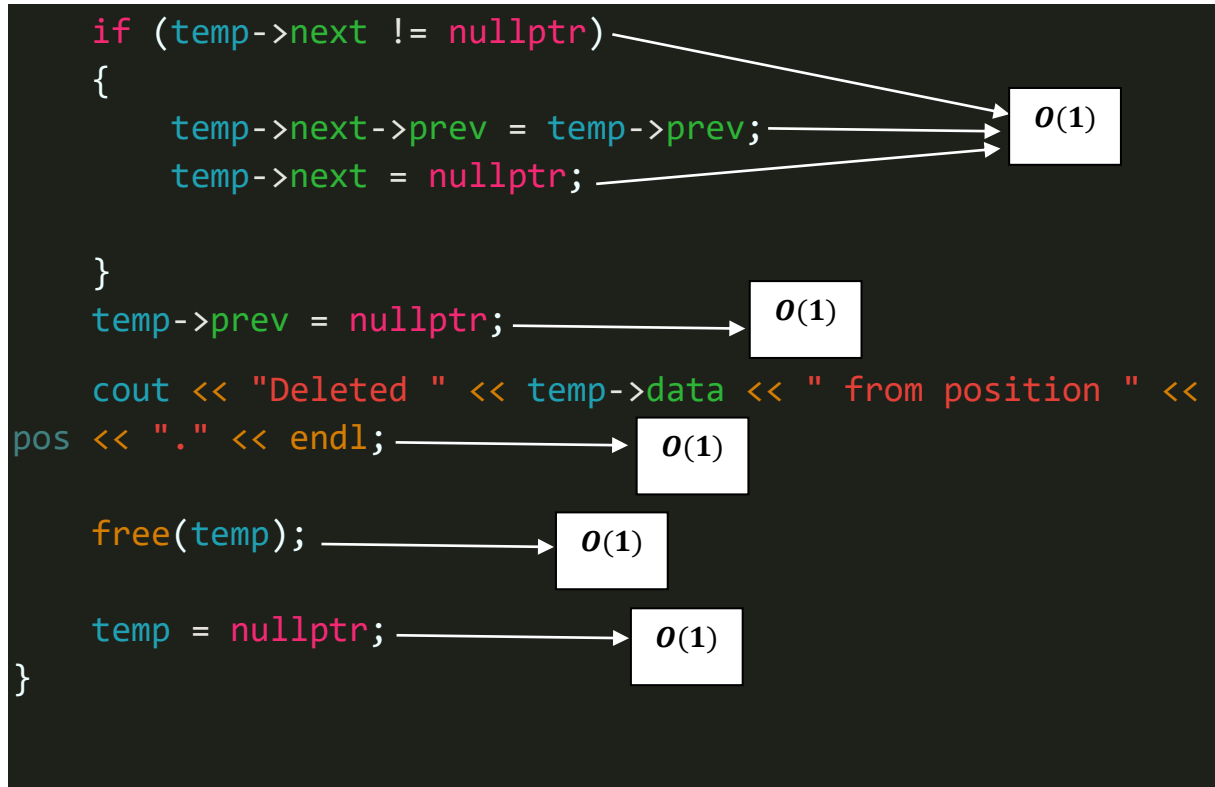
Time Complexity of Delete From Position Function Module

```
void deleteFromPosition(int pos)
{
    if (head == nullptr) →  $O(1)$ 
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1) →  $O(1)$ 
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1) →  $O(1)$ 
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head; →  $O(1)$ 
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next; →  $1 + 1 + 1 + \dots + (n - 1) = O(n - 1)$ 
        =  $O(n)$ 
    }
    if (temp == nullptr) →  $O(1)$ 
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    temp->prev->next = temp->next; →  $O(1)$ 
```

RUNS 'n - 1' times at Worst Case.



Absolute Best Cases:

1. When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements take constant time. Hence overall process takes, $O(1)$ time.

2. When Pos < 1:

→ If Pos < 1, this checking takes constant time. The cout and return statements take constant time. Hence overall process takes, $O(1)$ time.

Not Absolute Best Case (When Pos = 1)

→ if $pos = 1$, it runs `deleteFromBeginning()` function, and function's time complexity is constant $\rightarrow O(1)$ and returns make exit from function, also takes constant $\rightarrow O(1)$, Hence total complexity is constant $\rightarrow O(1)$.

Why not Absolute Best Case?

Ans: As we know `deleteFromBeginning()` function have lots of line of codes needs to be processed, though constant but unlike $pos < 1$ and Empty List, which have less line of codes to get processed.

Worst Case

→ If $head = nullptr$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ If $pos < 1$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ if $pos = 1$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer pointing to, this process takes constant amount of time i.e. $O(1)$.

→ The loop runs from 1 and runs upto $(pos - 1)$, now if $pos = n$, then the loop runs $n - 1$ times, therefore loop statement: `temp = temp → next` runs $1 + 1 + \dots (n - 1)$ times = $O(n - 1)$ times = $O(n)$ time complexity.

→ if $temp = NULLPTR$, means during traversal it went Out of Bound and return statement makes function exit. This check takes constant amount of time i.e. $O(1)$.

→ Next pointer of Previous Node pointed by prev pointer of memory location pointed by temp pointer assigned to next pointer of memory location pointed by temp pointer i.e. assigned with `NULLPTR` making next pointer detaches to the memory location(last node), next pointer pointed to, also making previous node the last node. This assignment takes constant amount of time i.e. $O(1)$.

→ if checks `temp != NULLPTR` which is false as next pointer of last node carries `NULLPTR`, this check takes constant amount of time i.e. $O(1)$.

→ prev pointer pointed by temp pointer i.e. last node set to `NULLPTR`, hence prev pointer gets detached with previous node $(n - 1)^{th}$, this operation takes constant constant amount of time i.e. $O(1)$.

→ "Deleted data from position" is printed takes constant amount of time i.e. $O(1)$.

→ `free(temp)` frees the memory location that temp is pointing to, this process takes constant amount of time i.e. $O(1)$.

→ temp pointer is assigned to `nullptr` to avoid dangling pointer, this process takes constant amount of time i.e. $O(1)$.

Hence, total time complexity : $O(1) + O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$ time complexity.

Deleted position from 2 to N, while loop runs from 1 to N – 1.

And when the position is uniformly distributed from 1 to N:

(Average Case i. e. Undetermined Position)

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 time.

.

.

→ If the position is (N), the loop's inner statement runs (N – 1) times.

As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilites then , we take the best case also i. e.

$O(1)$, when position = 1 . Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 1 \right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N.

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant ' $\frac{1}{2}$ ', we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Delete At Position</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case	$\Theta(n)$
